

N. Shree Deepthi Batch – 37

AI-ASSISTED CODING ASSIGNMENT 23.2

Task 1 – Project Proposal

Students choose a real-world problem to solve with an AI-assisted application (e.g., e-commerce billing system, health monitoring dashboard, student attendance tracker).

• Prepare a project proposal including:

o Objective

o Features

o Tools/technologies to be used

o Expected output

Prompt Used-

Generate a complete project proposal for an AI-powered web application that solves a real-world problem. Include the following sections: project title, objective, key features, tools and technologies (frontend, backend, database, AI tools), and expected output. Keep it suitable for a college capstone project and make it innovative but feasible.

```

1 from dataclasses import dataclass
2 from typing import List
3
4 @dataclass
5 class ProjectProposal:
6     title: str
7     objective: str
8     key_features: List[str]
9     frontend_tech: List[str]
10    backend_tech: List[str]
11    database: str
12    ai_tools: List[str]
13    expected_output: str
14
15    def generate_report(self):
16        report = f"""
17PROJECT PROPOSAL
18=====
19Title: {self.title}
20Objective: {self.objective}
21
22Key Features:
23{chr(10).join(f" • {feature}" for feature in self.key_features)}
24
25Technologies:
26    Frontend: {' , '.join(self.frontend_tech)}

```

```

    Frontend: {' , '.join(self.frontend_tech)}
    Backend: {' , '.join(self.backend_tech)}
    Database: {self.database}
    AI/ML Tools: {' , '.join(self.ai_tools)}

Expected Output:
{self.expected_output}
"""
        return report

# Example usage
proposal = ProjectProposal(
    title="AI-Powered Study Assistant",
    objective="Help students learn efficiently using personalized AI tutoring",
    key_features=["Personalized learning paths", "Real-time Q&A", "Progress tracking"],
    frontend_tech=["React", "Tailwind CSS"],
    backend_tech=["FastAPI", "Python"],
    database="PostgreSQL",
    ai_tools=["OpenAI API", "Scikit-learn"],
    expected_output="Web app with 80%+ user satisfaction"
)

print(proposal.generate_report())

```

PROJECT PROPOSAL

=====

Title: AI-Powered Study Assistant

Objective: Help students learn efficiently using personalized AI tutoring

Key Features:

- Personalized learning paths
- Real-time Q&A
- Personalized learning paths
- Personalized learning paths
- Real-time Q&A
- Progress tracking
- Personalized learning paths
- Real-time Q&A
- Personalized learning paths
- Personalized learning paths
- Personalized learning paths
- Personalized learning paths
- Real-time Q&A
- Personalized learning paths
- Real-time Q&A
- Progress tracking

Technologies:

Frontend: React, Tailwind CSS

Technologies:

Frontend: React, Tailwind CSS

Backend: FastAPI, Python

Database: PostgreSQL

AI/ML Tools: OpenAI API, Scikit-learn

Frontend: React, Tailwind CSS

Backend: FastAPI, Python

Database: PostgreSQL

AI/ML Tools: OpenAI API, Scikit-learn

Backend: FastAPI, Python

Database: PostgreSQL

AI/ML Tools: OpenAI API, Scikit-learn

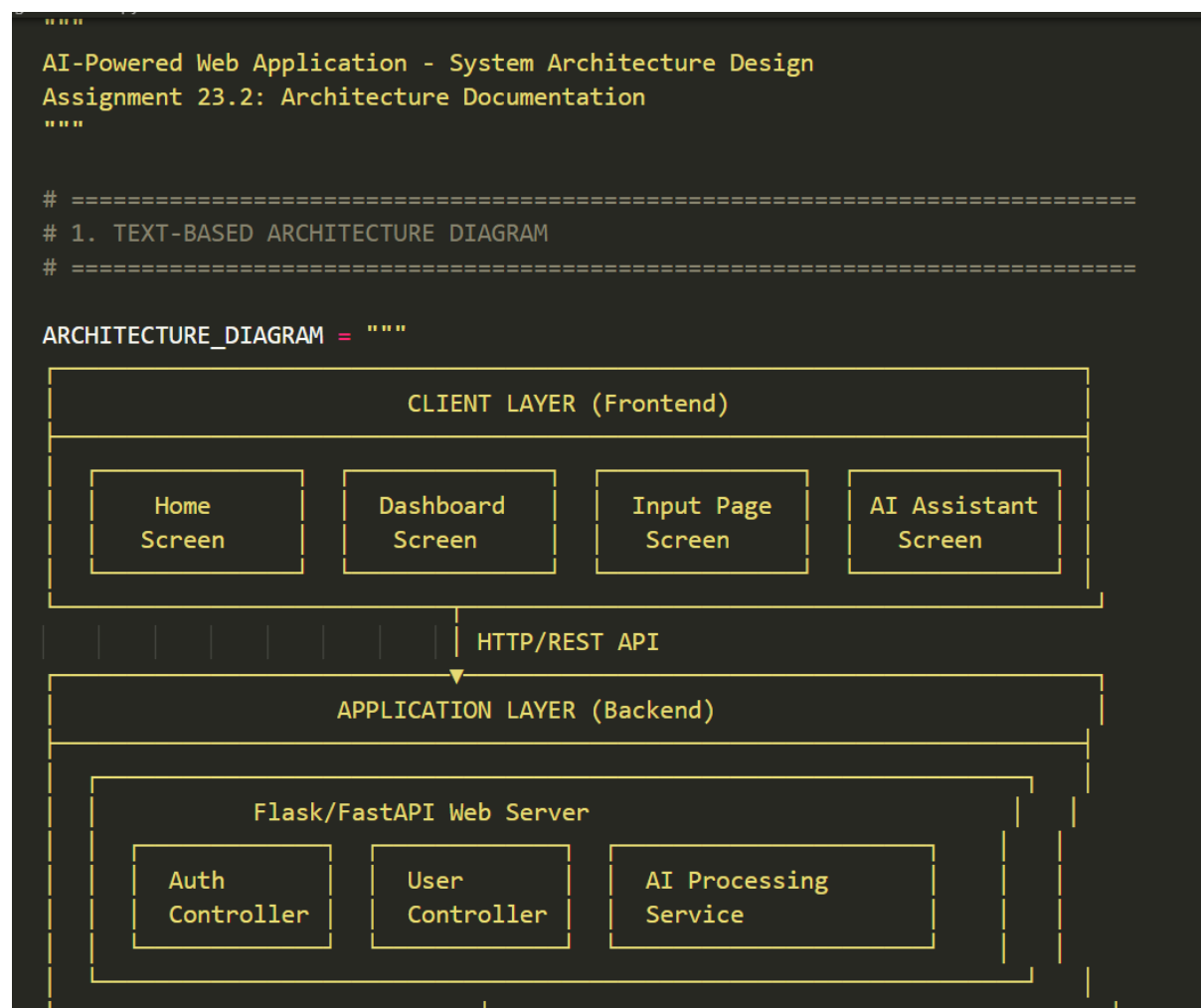
Explanation:

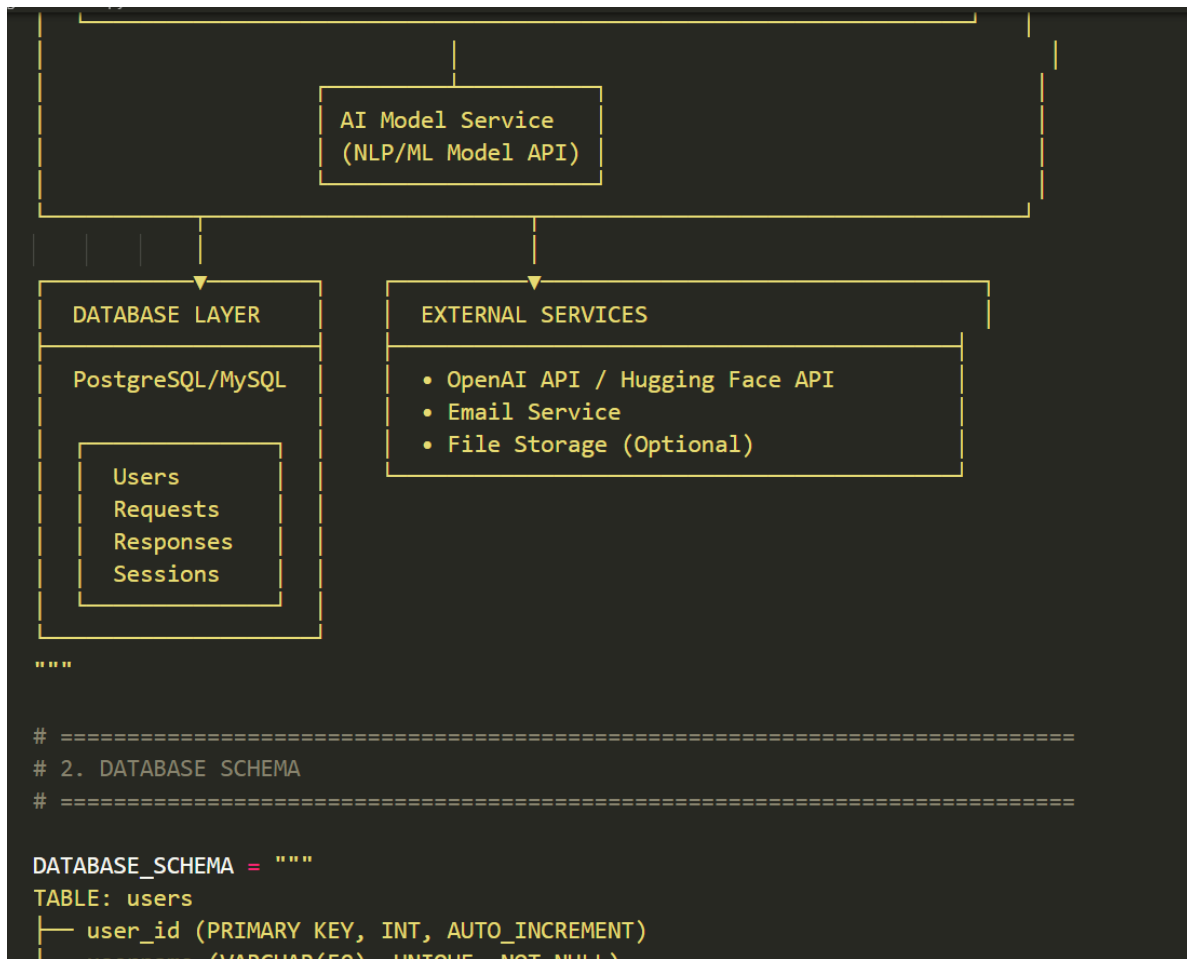
Task 2 – Application Design

- Use AI tools to assist in:
 - o Designing system architecture diagrams.
 - o Defining database schema (if needed).
 - o Creating wireframes/UI mockups for frontend design.

Prompt used:

Design the system architecture for an AI-powered web application. Include a clear architecture diagram (text-based), database schema with tables and attributes, and basic UI wireframe descriptions for each screen (home, dashboard, input page, AI assistant). Keep the design simple, scalable, and suitable for student-level implementation.





```
DATABASE_SCHEMA = ""
```

```
TABLE: users
```

```
|— user_id (PRIMARY KEY, INT, AUTO_INCREMENT)
|— username (VARCHAR(50), UNIQUE, NOT NULL)
|— email (VARCHAR(100), UNIQUE, NOT NULL)
|— password_hash (VARCHAR(255), NOT NULL)
|— full_name (VARCHAR(100))
|— created_at (TIMESTAMP, DEFAULT CURRENT_TIMESTAMP)
|— updated_at (TIMESTAMP, DEFAULT CURRENT_TIMESTAMP)
|— is_active (BOOLEAN, DEFAULT TRUE)
```

```
TABLE: requests
```

```
|— request_id (PRIMARY KEY, INT, AUTO_INCREMENT)
|— user_id (FOREIGN KEY → users.user_id)
|— input_text (TEXT, NOT NULL)
|— request_type (VARCHAR(50)) [e.g., 'analysis', 'summary', 'question']
|— created_at (TIMESTAMP, DEFAULT CURRENT_TIMESTAMP)
|— status (VARCHAR(20), DEFAULT 'pending') [pending, processing, completed]
```

```
TABLE: responses
```

```
|— response_id (PRIMARY KEY, INT, AUTO_INCREMENT)
|— request_id (FOREIGN KEY → requests.request_id, UNIQUE)
|— user_id (FOREIGN KEY → users.user_id)
|— output_text (TEXT, NOT NULL)
|— confidence_score (DECIMAL(3,2)) [0.00 to 1.00]
|— processing_time_ms (INT)
|— created_at (TIMESTAMP, DEFAULT CURRENT_TIMESTAMP)
|— model_used (VARCHAR(50))
```

UI_WIREFRAMES = ""



SCREEN 2: DASHBOARD (After Login)

HEADER: Logo | Dashboard | Messages | Profile | Logout

SIDEBAR

- Home
- New Query
- History
- Settings
- Help

MAIN CONTENT AREA

Welcome, [Username]!

STATS CARDS:

Total Queries: 25

Avg Response Time: 2.3s

Accuracy: 92%

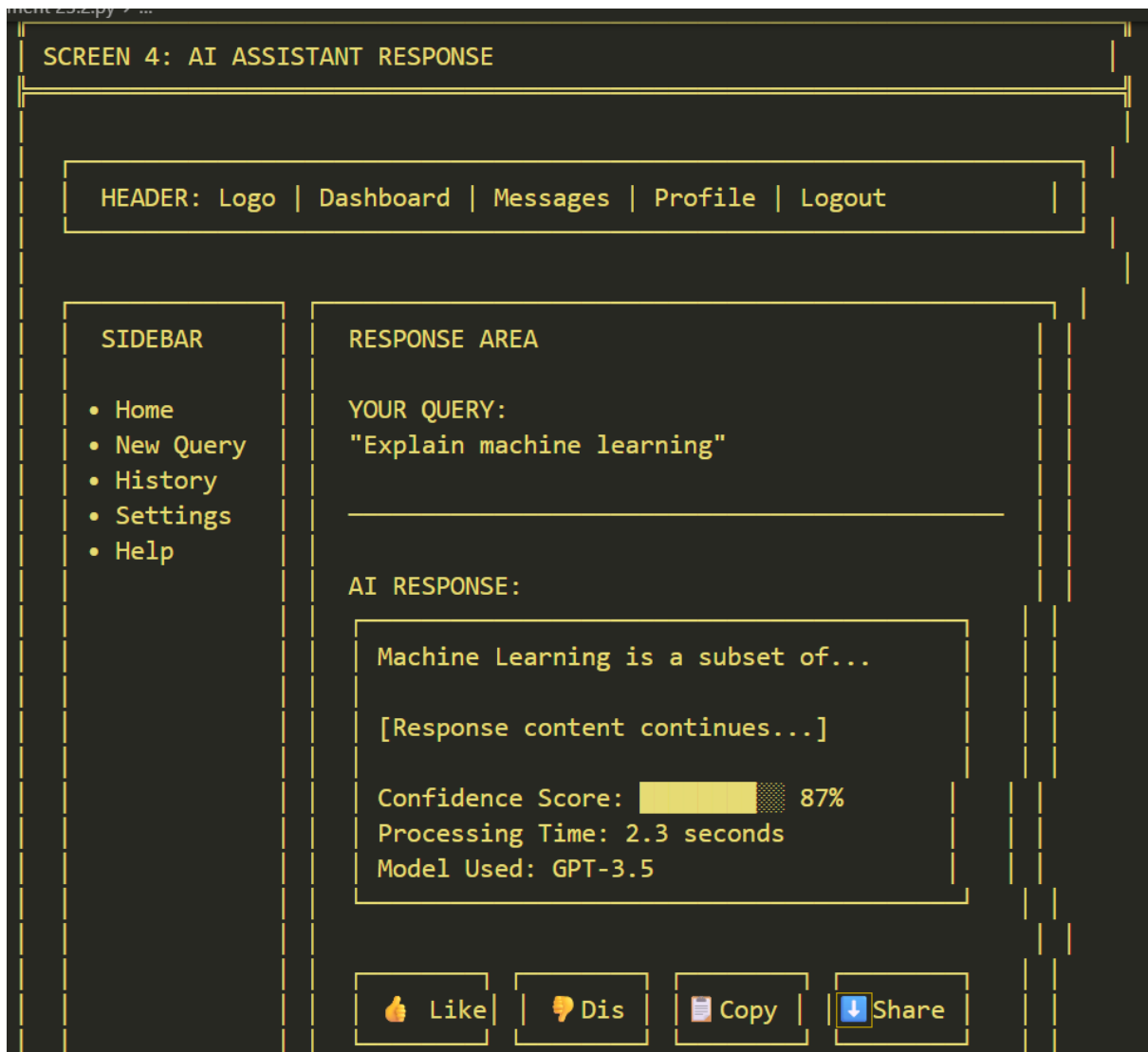
RECENT QUERIES:

[Query 1 Preview] - 2 hours ago

[Query 2 Preview] - Yesterday

[Query 3 Preview] - 2 days ago

[View All History]



```
API_ENDPOINTS = ""
```

Authentication:

POST	/api/auth/register	- Register new user
POST	/api/auth/login	- User login
POST	/api/auth/logout	- User logout
POST	/api/auth/refresh-token	- Refresh JWT token

User Management:

GET	/api/users/profile	- Get user profile
PUT	/api/users/profile	- Update user profile
GET	/api/users/settings	- Get user settings
PUT	/api/users/settings	- Update user settings

AI Queries:

POST	/api/queries	- Submit new query
GET	/api/queries	- Get user's queries
GET	/api/queries/{query_id}	- Get specific query
POST	/api/queries/{query_id}/follow-up	- Submit follow-up

Responses:

GET	/api/responses/{response_id}	- Get AI response
POST	/api/responses/{response_id}/feedback	- Submit feedback
POST	/api/responses/{response_id}/export	- Export response

Admin:

GET	/api/admin/stats	- Get system statistics
GET	/api/admin/users	- Get all users
POST	/api/admin/model-config	- Configure AI models

```
""
```

```

9
10 TECH_STACK = {
11     "Frontend": [
12         "HTML5 / CSS3 / JavaScript (Vanilla or React)",
13         "Bootstrap or Tailwind CSS for styling",
14         "Fetch API or Axios for HTTP requests"
15     ],
16     "Backend": [
17         "Python (Flask or FastAPI) - RESTful API",
18         "Flask-RESTful / FastAPI Routing",
19         "JWT for authentication",
20         "Python-dotenv for configuration"
21     ],
22     "Database": [
23         "PostgreSQL or MySQL",
24         "SQLAlchemy ORM for database operations"
25     ],
26     "AI/ML": [
27         "OpenAI API / Hugging Face Transformers",
28         "Or: sqlite3 with local LLM"
29     ],
30     "Deployment": [
31         "Docker for containerization",
32         "Heroku / AWS / Azure for hosting",
33         "GitHub for version control"
34     ]
35 }
36

```

```

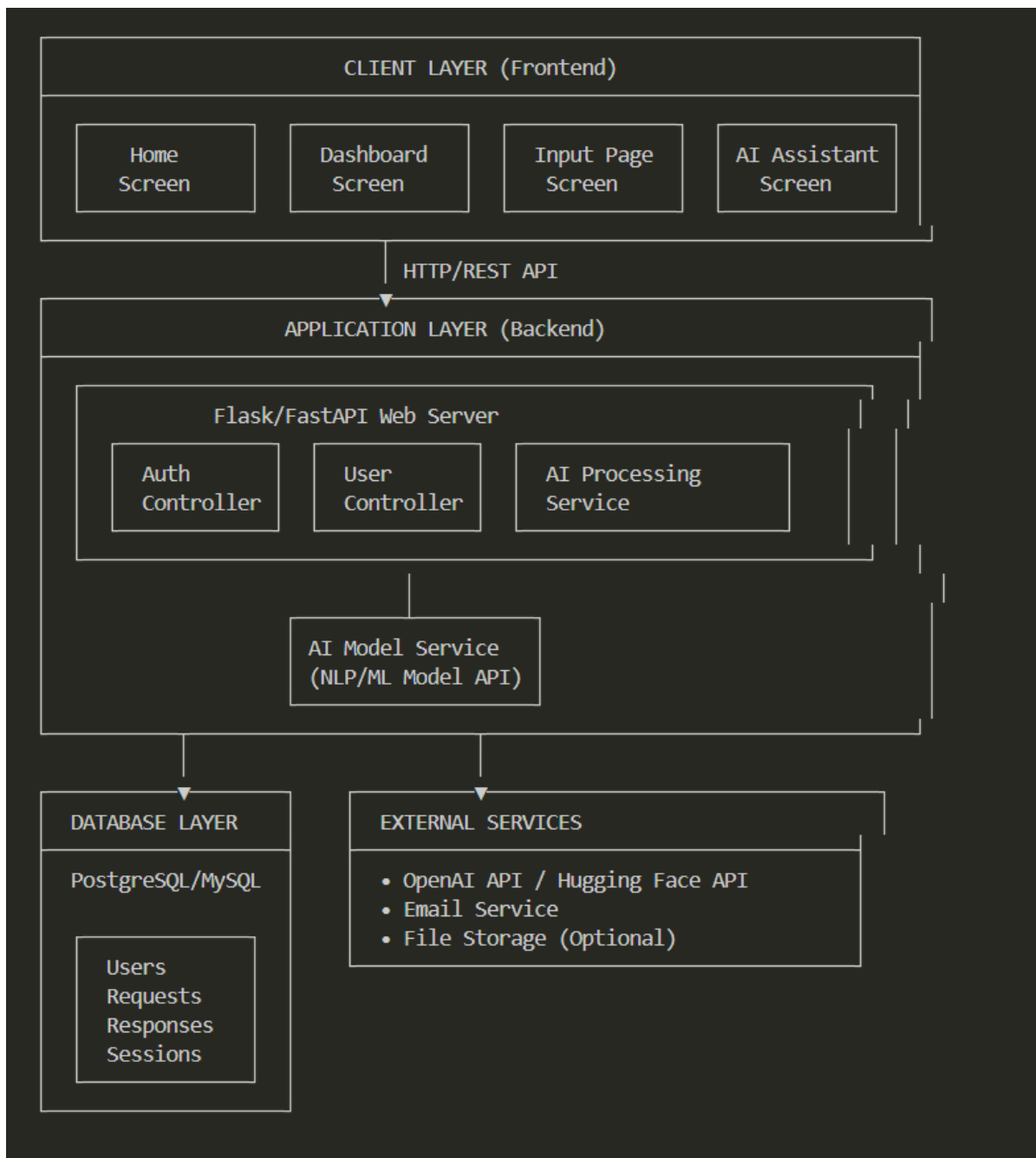
# =====
# 6. SIMPLE IMPLEMENTATION SKELETON
# =====

```

```

print(ARCHITECTURE_DIAGRAM)
print("\n" + "="*75 + "\n")
print(DATABASE_SCHEMA)
print("\n" + "="*75 + "\n")
print(UI_WIREFRAMES)

```



```

— user_id (PRIMARY KEY, INT, AUTO_INCREMENT)
— username (VARCHAR(50), UNIQUE, NOT NULL)
— email (VARCHAR(100), UNIQUE, NOT NULL)
— password_hash (VARCHAR(255), NOT NULL)
— full_name (VARCHAR(100))
— created_at (TIMESTAMP, DEFAULT CURRENT_TIMESTAMP)
— updated_at (TIMESTAMP, DEFAULT CURRENT_TIMESTAMP)
— is_active (BOOLEAN, DEFAULT TRUE)

TABLE: requests
— request_id (PRIMARY KEY, INT, AUTO_INCREMENT)
— user_id (FOREIGN KEY → users.user_id)
— input_text (TEXT, NOT NULL)
— request_type (VARCHAR(50)) [e.g., 'analysis', 'summary', 'question']
— created_at (TIMESTAMP, DEFAULT CURRENT_TIMESTAMP)
— status (VARCHAR(20), DEFAULT 'pending') [pending, processing, completed]

TABLE: responses
— response_id (PRIMARY KEY, INT, AUTO_INCREMENT)
— request_id (FOREIGN KEY → requests.request_id, UNIQUE)
— user_id (FOREIGN KEY → users.user_id)
— output_text (TEXT, NOT NULL)
— confidence_score (DECIMAL(3,2)) [0.00 to 1.00]
— processing_time_ms (INT)
— created_at (TIMESTAMP, DEFAULT CURRENT_TIMESTAMP)
— model_used (VARCHAR(50))

TABLE: sessions
— session_id (PRIMARY KEY, VARCHAR(255), UNIQUE)
— user_id (FOREIGN KEY → users.user_id)
— token (VARCHAR(500), NOT NULL)
— created_at (TIMESTAMP, DEFAULT CURRENT_TIMESTAMP)
— expires_at (TIMESTAMP)
— is_active (BOOLEAN, DEFAULT TRUE)

```

Explanation-

Task 3 – AI-Assisted Code Development

- Implement the application in phases:
 - o Frontend: Web/mobile UI with HTML/CSS/JS (or framework).
 - o Backend: RESTful APIs or server logic.
 - o Database: Schema design, SQL queries.
 - o Integration: API calls, authentication, error handling.

Prompt Used:

Generate complete starter code for an AI-powered web application including frontend (HTML, CSS, JavaScript), backend (Flask or Node.js API), and database integration. Include API routes, sample UI, and basic AI chatbot integration using an API. Ensure the code is modular, well-structured, and beginner-friendly.

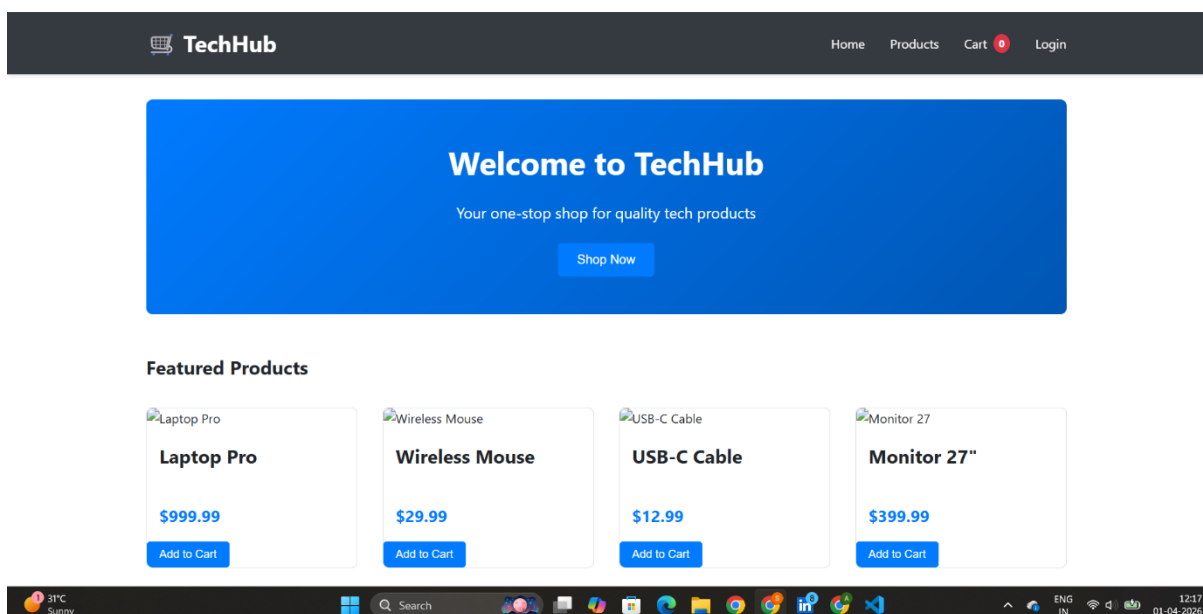
```
SETUP_GUIDE.md IMPLEMENTATION_GUIDE.md database.sql ecommerce.html X ...
ecommerce.html > html > body
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4     <meta charset="UTF-8">
5     <meta name="viewport" content="width=device-width, initial-scale=1">
6     <title>TechHub - E-Commerce Store</title>
7     <link rel="stylesheet" href="ecommerce.css">
8 </head>
9 <body>
10     <!-- Navigation -->
11     <nav class="navbar">
12         <div class="container">
13             <div class="nav-brand">
14                 <h1>🛒 TechHub</h1>
15             </div>
16             <ul class="nav-menu">
17                 <li><a href="#" class="nav-link" onclick="showHome()">Home</a>
18                 <li><a href="#" class="nav-link" onclick="showProducts()">Products</a>
19                 <li><a href="#" class="nav-link" onclick="showCart()">Cart</a>
20                 <li id="authNav">
21                     <a href="#" class="nav-link" onclick="showAuth()">Auth</a>
22                 </li>
23             </ul>
24         </div>
25     </nav>
26
27     <!-- Main Container -->
28     <div class="container">
29         <!-- Home Section -->
```

```

1 SQLite format 3
2
3
4
5 CREATE TABLE order_items (
6     id INTEGER PRIMARY KEY AUTOINCREMENT,
7     order_id INTEGER NOT NULL,
8     product_id INTEGER NOT NULL,
9     quantity INTEGER NOT NULL,
10    price REAL NOT NULL,
11    FOREIGN KEY (order_id) REFERENCES orders(id) ON DELETE CASCADE,
12    FOREIGN KEY (product_id) REFERENCES products(id)
13 )
14 CREATE TABLE orders (
15     id INTEGER PRIMARY KEY AUTOINCREMENT,
16     user_id INTEGER NOT NULL,
17     total_price REAL NOT NULL,
18     status TEXT DEFAULT 'pending',
19     shipping_address TEXT,
20     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
21     updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
22     FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
23 )
24 CREATE TABLE cart (
25     id INTEGER PRIMARY KEY AUTOINCREMENT,
26     user_id INTEGER NOT NULL,
27     product_id INTEGER NOT NULL,
28     quantity INTEGER NOT NULL DEFAULT 1,
29     added_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
30     FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
31     FOREIGN KEY (product_id) REFERENCES products(id) ON DELETE CASCADE
32 )
```

```
# ecommerce.css > ...
1  /* ===== GLOBAL STYLES ===== */
2
3  * {
4      margin: 0;
5      padding: 0;
6      box-sizing: border-box;
7  }
8
9  :root {
10     --primary-color: #007bff;
11     --secondary-color: #6c757d;
12     --success-color: #28a745;
13     --danger-color: #dc3545;
14     --dark-color: #343a40;
15     --light-color: #f8f9fa;
16     --border-color: #dee2e6;
17     --text-color: #212529;
18 }
19
20 body {
21     font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
22     line-height: 1.6;
23     color: var(--text-color);
24     background-color: #fff;
25 }
26
27 /* ===== NAVIGATION ===== */
28
29 .navbar {
```

```
database.sql
▶ Run on active connection | ≡ Select block
1  -- Database Schema for E-Commerce Application
2
3  -- Users table
4  CREATE TABLE IF NOT EXISTS users (
5      id INTEGER PRIMARY KEY AUTOINCREMENT,
6      username TEXT UNIQUE NOT NULL,
7      email TEXT UNIQUE NOT NULL,
8      password TEXT NOT NULL,
9      full_name TEXT,
10     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
11     updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
12 );
13
14 -- Products table
15 CREATE TABLE IF NOT EXISTS products (
16     id INTEGER PRIMARY KEY AUTOINCREMENT,
17     name TEXT NOT NULL,
18     description TEXT,
19     price REAL NOT NULL,
20     stock INTEGER NOT NULL DEFAULT 0,
21     category TEXT,
22     image_url TEXT,
23     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
24     updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
25 );
26
27 -- Shopping Cart table
28 CREATE TABLE IF NOT EXISTS cart (
29     id INTEGER PRIMARY KEY AUTOINCREMENT
```



Featured Products



Laptop Pro

Laptop Pro

\$999.99

Add to Cart



Wireless Mouse

Wireless Mouse

\$29.99

Add to Cart



USB-C Cable

USB-C Cable

\$12.99

Add to Cart



Monitor 27

Monitor 27"

\$399.99

Add to Cart



Keyboard Mechanical

Keyboard Mechanical

\$79.99

Add to Cart



Webcam HD

Webcam HD

\$49.99

Add to Cart

TechHub

HomeProductsCart0Login

Register

Full Name

sru

Username

sru

Email

sru@gmail.com

Password

...

Register

TechHub

HomeProductsCart0Login

Shopping Cart

Cart Summary

Subtotal:	\$0.00
Shipping:	\$10.00
Total:	\$10.00

Proceed to Checkout

Task 4 – Documentation & Code Quality

- **Generate automatic documentation and inline comments using**

AI.

- **Ensure clear README.md for project setup and usage.**
- **Conduct AI-assisted code review to refine readability and maintainability.**

Prompt used :

Create a complete README.md file for an AI-powered web application project. Include project description, features, technologies used, installation steps, usage instructions, and folder structure. Also add inline code comments and suggestions to improve code readability and maintainability.

```
def get_profile():
    """
    Retrieve authenticated user's profile information.

    GET /api/auth/profile

    Headers Required:
    |   Authorization: Bearer <JWT_TOKEN>

    Response (200 OK):
    |   {
    |       "id": 1,
    |       "username": "john_doe",
    |       "email": "john@example.com",
    |       "full_name": "John Doe",
    |       "created_at": "2024-01-15T10:30:00"
    |   }

    Errors:
    |   401: Missing or invalid JWT token
    |   404: User not found (shouldn't happen if token is valid)
    |   500: Server error
    """
```

```

102 def register():
132     data = request.get_json()
133
134     # Validate required fields
135     if not data or not data.get('username') or not data.get('email') or not data.get('password'):
136         return jsonify({'error': 'Missing required fields'}), 400
137
138     try:
139         db = get_db()
140         # Hash password for security - never store plain text
141         hashed_password = generate_password_hash(data['password'])
142
143         # Insert new user with hashed password
144         db.execute(
145             'INSERT INTO users (username, email, password, full_name) VALUES (
146                 (data['username'], data['email'], hashed_password,
147                 data['full_name'])
148             )
149         db.commit()
150
151         # Get the newly created user ID
152         user_id = db.execute('SELECT last_insert_rowid()').fetchone()[0]
153         db.close()
154
155         # Generate JWT token for immediate login after registration
156         access_token = create_access_token(identity=user_id)
157         return jsonify({'message': 'User registered successfully', 'token': access_token}), 201
158     except sqlite3.IntegrityError:
159         # Handles duplicate username or email constraint violation
160         return jsonify({'error': 'Username or email already exists'}), 400

```

Keep

Undo



2 of 45



```

1  # 🛒 TechHub - Full-Stack E-Commerce Application
2
3  **A modern, production-ready e-commerce platform built with
4  Python, JavaScript, and SQLite**
5
6  ![Status](https://img.shields.io/badge/Status-Production%20Ready-green)
7  ![Version](https://img.shields.io/badge/Version-1.0.0-orange)
8  ---
9
10 ## 📖 Overview
11
12 TechHub is a complete, production-grade e-commerce
13 application demonstrating full-stack development:
14
15 - **Frontend**: Responsive UI with HTML5, CSS3, Vanilla
16   JavaScript
17 - **Backend**: RESTful API built with Flask and Python
18 - **Database**: SQLite3 with proper relational schema
19 - **Authentication**: JWT-based token authentication
20 - **Features**: Product catalog, shopping cart, checkout,
21   order management, user authentication
22
23 **Perfect for**: Learning, portfolio projects, or as a
24 foundation for real applications.
25
26 ---

```

1 # 🛒 TechHub - Full-Stack E-Commerce Application

24 ## ✨ Key Features

25

26 ### 👤 User Management

- 27 - User registration with validation
- 28 - Secure login with JWT tokens
- 29 - Password hashing with Werkzeug
- 30 - Session persistence
- 31 - Automatic logout on token expiration

32

33 ### 📦 Product Management

- 34 - Dynamic product catalog
- 35 - Full-text search functionality
- 36 - Category-based filtering
- 37 - Real-time stock tracking
- 38 - Product modal previews

39

40 ### 🛒 Shopping Features

- 41 - Add/remove items from cart
- 42 - Adjust quantities with validation
- 43 - Real-time price calculations
- 44 - Stock inventory verification
- 45 - Multi-step checkout process

46

47 ### 📋 Order Management

- 48 - Order history tracking
- 49 - Order status management
- 50 - Shipping address storage
- 51 - Order details with itemization

📄 Code Quality & Best Practices Guide

Overview

This document outlines the code quality standards, architecture decisions, and best practices implemented in TechHub.

🏗️ Architecture Principles

1. Separation of Concerns

- ****Backend****: Handles data logic, authentication, validation
- ****Frontend****: Manages UI, user interactions, state
- ****Database****: Persists data with proper relationships
- Clear boundaries between layers

2. RESTful API Design

- Standard HTTP methods (GET, POST, PUT, DELETE)
- Consistent endpoint naming conventions
- Proper HTTP status codes (200, 201, 400, 401, 404, 500)
- JSON request/response format

3. Security First

- Passwords hashed before storage
- JWT tokens for stateless authentication
- Parameterized SQL queries prevent injection
- Input validation on both frontend and backend

```
backend.py > ...
1  """
2
3      TechHub E-Commerce REST API - Backend Server
4
5      A complete RESTful API for an e-commerce platform with:
6      - User authentication (JWT tokens)
7      - Product management and catalog
8      - Shopping cart operations
9      - Order processing and tracking
10
11      Framework: Flask with JWT authentication
12      Database: SQLite3
13      Author: AI-Assisted Development
14      Version: 1.0.0
15
16  """
17
18  # ----- IMPORTS -----
```

Task 5 – Testing & Debugging

- Use AI to:
 - o Generate unit tests.
 - o Identify vulnerabilities/security flaws.
 - o Suggest optimizations for performance.

Prompt used-

Generate unit tests and debugging strategies for an AI-powered web application. Include sample test cases for frontend and backend, identify potential bugs and security vulnerabilities, and suggest performance optimizations. Keep it simple and suitable for student-level projects.

TechHub Backend Unit Tests

Comprehensive test suite for Flask REST API using pytest.
Tests cover: authentication, products, cart, orders, error

Run tests with: `pytest test_backend.py -v`

```
"""

import pytest
import json
import sqlite3
from datetime import timedelta
from werkzeug.security import generate_password_hash, check_password_hash

# Import Flask app and functions
import sys
import os
sys.path.insert(0, os.path.dirname(__file__))

# Test database setup
import tempfile
import backend

@pytest.fixture
def client():
    """
```

```

28 def client():
39     db_fd, db_path = tempfile.mkstemp()
40     # Configure Flask for testing
41     backend.app.config['TESTING'] = True
42     backend.app.config['JWT_SECRET_KEY'] = 'test-secret-key'
43     backend.DATABASE = db_path
44
45     # Initialize test database
46     with backend.app.app_context():
47         backend.init_db()
48
49     # Create test client
50     client = backend.app.test_client()
51
52     yield client
53
54     # Cleanup
55     os.close(db_fd)
56     os.unlink(db_path)
57
58 @pytest.fixture
59 def auth_token(client):
60     """
61     Fixture providing authenticated JWT token.
62
63     Creates test user and returns valid JWT token.
64     Usage: def test_something(auth_token): ...
65     """

```

Task 6 – Deployment & Presentation

- Deploy application on a cloud/free hosting platform (Heroku, GitHub Pages, Firebase, etc.).
- Prepare a final presentation including:
 - o Problem statement
 - o Solution architecture
 - o Live demo
 - o Reflection on AI assistance and limitations

prompt used-

Provide a step-by-step guide to deploy an AI-powered web application using platforms like GitHub Pages, Firebase, or Heroku. Also generate a presentation outline including problem statement, solution, architecture, demo, and conclusion. Keep it concise and easy to present in a classroom.

```
1  # TechHub E-commerce Application - Docker Configuration
2
3  # Use Python 3.9 slim image for smaller size
4  FROM python:3.9-slim
5
6  # Set working directory
7  WORKDIR /app
8
9  # Install system dependencies
10 RUN apt-get update && apt-get install -y \
11     gcc \
12     && rm -rf /var/lib/apt/lists/*
13
14 # Copy requirements first for better caching
15 COPY requirements.txt .
16
17 # Install Python dependencies
18 RUN pip install --no-cache-dir -r requirements.txt
19
20 # Copy application code
21 COPY . .
22
23 # Create non-root user for security
24 RUN useradd --create-home --shell /bin/bash app \
25     && chown -R app:app /app
26 USER app
27
28 # Expose port
29 EXPOSE 5000
```